

MNIST Neural Network & Backpropagation

Conversation transcript and mathematical explanation — focused on the NumPy-from-scratch neural network.

1. Original MNIST neural-network code

```
"""
A neural network that recognizes handwritten digits (0-9), built from
scratch with numpy -- no deep learning frameworks.

Notation used throughout:
    m = number of images in a batch (e.g. 128)
    784 = pixels per image (28 x 28 flattened into one row)
    W = weight matrix, b = bias vector
    Z = raw weighted sum (before activation)
    A = activated output (after ReLU or softmax)

Shape convention: every batch of data is a matrix of shape (m, features).
Each ROW is one image. This is the standard convention (as opposed to
each image being a column) and it's why some formulas below use W.T
(transpose) to make the matrix multiplication line up.
"""

import numpy as np

def load_mnist():
    from sklearn.datasets import fetch_openml
    print("Downloading MNIST (only happens once, then it's cached)...")
    mnist = fetch_openml('mnist_784', version=1, as_frame=False)
    X, y = mnist.data, mnist.target.astype(int)

    X_train, X_test = X[:60000], X[60000:]
    y_train, y_test = y[:60000], y[60000:]
    return X_train, y_train, X_test, y_test

def preprocess(X, y, num_classes=10):
    X_scaled = X / 255.0
    Y_onehot = np.eye(num_classes)[y]
    return X_scaled, Y_onehot

class SimpleMNISTNet:
    def __init__(self, learning_rate=0.05, l2_strength=1e-4):
        self.lr = learning_rate
        self.reg = l2_strength
        self.layer_sizes = [784, 256, 128, 10]
        self.num_layers = len(self.layer_sizes) - 1
        self._init_weights()

    def _init_weights(self):
        self.weights = []
        self.biases = []
        for i in range(self.num_layers):
            fan_in = self.layer_sizes[i]
            fan_out = self.layer_sizes[i + 1]
            W = np.random.randn(fan_out, fan_in) * np.sqrt(2.0 / fan_in)
            b = np.zeros((1, fan_out))
            self.weights.append(W)
            self.biases.append(b)

    def relu(self, Z):
        return np.maximum(0, Z)

    def relu_derivative(self, Z):
        return (Z > 0).astype(float)

    def softmax(self, Z):
        Z_shifted = Z - np.max(Z, axis=1, keepdims=True)
        exp_scores = np.exp(Z_shifted)
        return exp_scores / np.sum(exp_scores, axis=1, keepdims=True)

    def forward(self, X):
        self.cache = {"A0": X}
        A = X

        for i in range(self.num_layers):
```

```

        W, b = self.weights[i], self.biases[i]
        Z = A @ W.T + b
        is_last_layer = (i == self.num_layers - 1)
        A = self.softmax(Z) if is_last_layer else self.relu(Z)

        self.cache[f"Z{i+1}"] = Z
        self.cache[f"A{i+1}"] = A

    return A

def compute_loss(self, predictions, targets):
    m = targets.shape[0]
    data_loss = -np.sum(targets * np.log(predictions + 1e-8)) / m
    weight_penalty = sum(np.sum(W ** 2) for W in self.weights)
    l2_loss = (self.reg / (2 * m)) * weight_penalty
    return data_loss + l2_loss

def backward(self, targets):
    m = targets.shape[0]
    grads_W = [None] * self.num_layers
    grads_b = [None] * self.num_layers

    dZ = self.cache[f"A{self.num_layers}"] - targets

    for i in reversed(range(self.num_layers)):
        A_prev = self.cache[f"A{i}"]
        W = self.weights[i]

        grads_W[i] = (dZ.T @ A_prev) / m + (self.reg / m) * W
        grads_b[i] = np.sum(dZ, axis=0, keepdims=True) / m

        if i > 0:
            Z_prev = self.cache[f"Z{i}"]
            dZ = (dZ @ W) * self.relu_derivative(Z_prev)

    return grads_W, grads_b

def update(self, grads_W, grads_b):
    for i in range(self.num_layers):
        self.weights[i] -= self.lr * grads_W[i]
        self.biases[i] -= self.lr * grads_b[i]

def train(self, X_train, Y_train, X_test, Y_test, epochs=20, batch_size=128):
    m = X_train.shape[0]
    for epoch in range(1, epochs + 1):
        shuffled_idx = np.random.permutation(m)
        for start in range(0, m, batch_size):
            batch_idx = shuffled_idx[start:start + batch_size]
            X_batch = X_train[batch_idx]
            Y_batch = Y_train[batch_idx]

            self.forward(X_batch)
            grads_W, grads_b = self.backward(Y_batch)
            self.update(grads_W, grads_b)

        train_acc = self.accuracy(X_train, Y_train)
        test_acc = self.accuracy(X_test, Y_test)
        loss = self.compute_loss(self.forward(X_train), Y_train)

        print(f"Epoch {epoch:2d} | Loss {loss:.4f} | "
              f"Train {train_acc:.2f}% | Test {test_acc:.2f}%")

def predict(self, X):
    return np.argmax(self.forward(X), axis=1)

def accuracy(self, X, Y):
    predictions = self.predict(X)
    true_labels = np.argmax(Y, axis=1)
    return np.mean(predictions == true_labels) * 100

if __name__ == "__main__":
    X_train_raw, y_train_raw, X_test_raw, y_test_raw = load_mnist()
    X_train, Y_train = preprocess(X_train_raw, y_train_raw)
    X_test, Y_test = preprocess(X_test_raw, y_test_raw)

    print(f"X_train shape: {X_train.shape}")
    print(f"Y_train shape: {Y_train.shape}")

    net = SimpleMNISTNet(learning_rate=0.05, l2_strength=1e-4)
    net.train(X_train, Y_train, X_test, Y_test, epochs=20, batch_size=128)

```

2. What is `targets`?

`targets` is the matrix containing the correct answers for the batch.

For MNIST, each image has one label from 0 to 9. After one-hot encoding, a label such as 7 becomes:

```
[0, 0, 0, 0, 0, 0, 0, 1, 0, 0]
```

For a batch of m images:

```
targets.shape = (m, 10)
```

Mathematically, targets is usually written Y . It is the ground-truth matrix used to compare the network's predictions against the correct labels.

In the backward pass, the key expression is:

```
dZ3 = A3 - Y
```

where $A3$ contains predicted probabilities and Y contains the true one-hot labels.

3. What does `is\_last\_layer = (i == self.num\_layers - 1)` mean?

`self.num\_layers` is 3 because the network has three sets of weights:

W1: 784 -> 256

W2: 256 -> 128

W3: 128 -> 10

Python indexes them as 0, 1, 2. Therefore the last layer has index:

```
self.num_layers - 1 = 2
```

So:

```
is_last_layer = (i == 2)
```

returns True only when the loop is processing the output layer.

The line:

```
A = self.softmax(Z) if is_last_layer else self.relu(Z)
```

therefore means:

```
if this is the output layer:
    A = softmax(Z)
otherwise:
    A = ReLU(Z)
```

This is necessary because the hidden layers use ReLU, while the final layer must produce ten probabilities that sum to one.

4. Forward-pass cache and matrix shapes

The forward pass starts with:

```
self.cache = {"A0": X}
A = X
```

$A0$ is the input. For a batch of m images:

$A0 = X$ has shape $(m, 784)$

For layer 1:

```
Z1 = A0 W1^T + b1
```

Shapes:

A0: (m, 784)
W1^T: (784, 256)
b1: (1, 256)
Z1: (m, 256)

Then:

A1 = ReLU(Z1)

For layer 2:

Z2 = A1 W2^T + b2
A2 = ReLU(Z2)

For layer 3:

Z3 = A2 W3^T + b3
A3 = softmax(Z3)

Finally A3 has shape (m, 10), containing the predicted probability of each digit.

5. Backpropagation — mathematical meaning

Backpropagation calculates derivatives of the loss with respect to every trainable parameter.

The central meanings are:

$dZ = \partial L / \partial Z$
 $dW = \partial L / \partial W$
 $db = \partial L / \partial b$

The network is:

X -> Z1 -> A1 -> Z2 -> A2 -> Z3 -> A3 -> L

Forward equations:

Z1 = X W1^T + b1
A1 = ReLU(Z1)

Z2 = A1 W2^T + b2
A2 = ReLU(Z2)

Z3 = A2 W3^T + b3
A3 = softmax(Z3)

The loss is cross-entropy:

$L = -(1/m) \sum_r \sum_k Y_{rk} \log(A3_{rk})$

Backpropagation then walks from L toward X using the chain rule.

6. Step 1 — output error: $dZ3 = A3 - targets$

The code:

```
dZ = self.cache[f"A{self.num_layers}"] - targets
```

becomes:

$dZ3 = A3 - Y$

for softmax combined with cross-entropy loss.

For one sample:

$L = -\sum_k Y_k \log(A_k)$

Softmax is:

$A_k = \exp(Z_k) / \sum_j \exp(Z_j)$

Combining the derivative of cross-entropy with the derivative of softmax gives the important simplification:

$$\partial L / \partial Z_3 = A_3 - Y$$

So ∂Z_3 is the error signal at the output layer.

Example:

True digit = 7

$Y = [0, 0, 0, 0, 0, 0, 1, 0, 0]$

Suppose:

$A_3 = [0.01, 0.02, 0.03, 0.04, 0.70, 0.02, 0.01, 0.10, 0.05, 0.02]$

Then:

$$dZ_3 = A_3 - Y$$

$= [0.01, 0.02, 0.03, 0.04, 0.70, 0.02, 0.01, -0.90, 0.05, 0.02]$

A negative value for the correct class indicates that increasing its pre-activation tends to reduce the loss.

7. Step 2 — why $dW = dZ^T A_{\text{prev}} / m$?

Consider:

$$Z_3 = A_2 W_3^T + b_3$$

For one weight w_{ji} :

$$\begin{aligned} \partial L / \partial w_{ji} &= (\partial L / \partial z_j) (\partial z_j / \partial w_{ji}) \end{aligned}$$

Since:

$$\partial L / \partial z_j = dZ_j$$

and:

$$\partial z_j / \partial w_{ji} = a_i$$

we obtain:

$$\partial L / \partial w_{ji} = dZ_j a_i$$

For a whole batch, matrix multiplication computes all such products simultaneously:

$$dZ_3^T A_2$$

Shapes:

$$\begin{aligned} dZ_3: & (m, 10) \\ dZ_3^T: & (10, m) \\ A_2: & (m, 128) \end{aligned}$$

Therefore:

$$(10, m)(m, 128) = (10, 128)$$

which is exactly W_3 's shape.

Because the loss is averaged over m examples, divide by m :

$$dW_3 = (dZ_3^T A_2) / m$$

With L2 regularization:

$$dW_3 = (dZ_3^T A_2) / m + (\lambda / m) W_3$$

8. Step 3 — bias gradient

The forward equation is:

$$z_j = \sum_i w_{ji} a_i + b_j$$

Therefore:

$$\partial z_j / \partial b_j = 1$$

so:

$$\partial L / \partial b_j = \partial L / \partial z_j = dZ_j$$

For a batch, average over all examples:

$$db = (1/m) \sum_r dZ_r$$

The NumPy line:

```
np.sum(dZ, axis=0, keepdims=True) / m
```

sums the gradient for each neuron across the batch.

9. Step 4 — propagate error to the previous layer

The forward equation is:

$$Z3 = A2 W3^T + b3$$

We first need:

$$dA2 = \partial L / \partial A2$$

By the chain rule:

$$dA2 = dZ3 W3$$

Why?

For one hidden activation $A2_i$:

$$\begin{aligned} \partial L / \partial A2_i &= \sum_j (\partial L / \partial Z3_j) (\partial Z3_j / \partial A2_i) \\ &= \sum_j dZ3_j W3_{ji} \end{aligned}$$

The matrix form is:

$$dA2 = dZ3 W3$$

Shapes:

$$\begin{aligned} dZ3: & (m, 10) \\ W3: & (10, 128) \end{aligned}$$

therefore:

$$dA2: (m, 128)$$

10. Step 5 — pass through ReLU

We have:

$$A2 = \text{ReLU}(Z2)$$

Therefore, by the chain rule:

$$dZ2 = dA2 \blacksquare \text{ReLU}'(Z2)$$

Substituting $dA2$:

$$dZ2 = (dZ3 W3) \blacksquare \text{ReLU}'(Z2)$$

This is exactly:

```
dZ = (dZ @ W) * self.relu_derivative(Z_prev)
```

ReLU is:

$\text{ReLU}(z) = \max(0, z)$

Its derivative is:

```
ReLU'(z) = 1, if z > 0
           0, if z <= 0
```

Example:

$Z_2 = [-2, 5, -1, 3]$

$\text{ReLU}'(Z_2) = [0, 1, 0, 1]$

If:

```
dA2 = [0.4, -0.7, 0.2, 0.5]
```

then:

```
dZ2 = [0, -0.7, 0, 0.5]
```

The zero entries occur because a negative pre-activation passes through the flat part of ReLU, whose derivative is zero.

11. Step 6 — repeat for layer 2 and layer 1

Once dZ_2 is known:

```
dW2 = (dZ2^T A1)/m + (λ/m)W2
```

```
db2 = (1/m)Σ dZ2
```

Then propagate:

```
dA1 = dZ2 W2
```

```
dZ1 = dA1 ■ ReLU'(Z1)
```

Therefore:

```
dZ1 = (dZ2 W2) ■ ReLU'(Z1)
```

Finally:

```
dW1 = (dZ1^T X)/m + (λ/m)W1
```

```
db1 = (1/m)Σ dZ1
```

There is no need to calculate dZ_0 because $A_0 = X$ is the input data, not a trainable layer.

12. The complete backpropagation equations

Output:

```
dZ3 = A3 - Y
```

Layer 3:

```
dW3 = (dZ3^T A2)/m + (λ/m)W3
```

```
db3 = (1/m)Σ dZ3
```

Propagate:

```
dZ2 = (dZ3 W3) ■ ReLU'(Z2)
```

Layer 2:

```
dW2 = (dZ2^T A1)/m + (λ/m)W2
```

$$db2 = (1/m) \sum dZ2$$

Propagate:

$$dZ1 = (dZ2 W2) \blacksquare \text{ReLU}'(Z1)$$

Layer 1:

$$dw1 = (dZ1^T X)/m + (\lambda/m)W1$$

$$db1 = (1/m) \sum dZ1$$

13. The central idea — the chain rule

Backpropagation is repeated application of the chain rule.

For W1:

$$\begin{aligned} &\partial L / \partial W1 \\ &= \\ &(\partial L / \partial A3) \\ &(\partial A3 / \partial Z3) \\ &(\partial Z3 / \partial A2) \\ &(\partial A2 / \partial Z2) \\ &(\partial Z2 / \partial A1) \\ &(\partial A1 / \partial Z1) \\ &(\partial Z1 / \partial W1) \end{aligned}$$

Rather than calculating this huge expression independently for every weight, backpropagation computes intermediate derivatives once and reuses them.

That is why the error signal moves backward:

$$dZ3 \rightarrow dZ2 \rightarrow dZ1$$

while the parameter gradients are collected:

$$\begin{aligned} &dw3, db3 \\ &dw2, db2 \\ &dw1, db1 \end{aligned}$$

The final update is gradient descent:

$$\begin{aligned} W_i &\leftarrow W_i - \eta \, dw_i \\ b_i &\leftarrow b_i - \eta \, db_i \end{aligned}$$

where η is the learning rate.

14. Backpropagation in one visual flow

FORWARD:

```

X
↓
Z1 = XW1^T + b1
↓
A1 = ReLU(Z1)
↓
Z2 = A1W2^T + b2
↓
A2 = ReLU(Z2)
↓
Z3 = A2W3^T + b3
↓
A3 = softmax(Z3)
↓
LOSS

```

BACKWARD:

```

LOSS
↓
dZ3 = A3 - Y
↓
dw3, db3
↓
dA2 = dZ3 W3
↓

```

```

dZ2 = dA2 ■ ReLU'(Z2)
↓
dw2, db2
↓
dA1 = dZ2 w2
↓
dZ1 = dA1 ■ ReLU'(Z1)
↓
dw1, db1

```

The whole learning cycle is:

Input → Prediction → Loss → Derivatives → Weight update → Better prediction.

Note: This PDF compiles the conversation content available in the current chat context. It preserves the original code and the detailed explanations developed in the visible exchange.